CineWorld Movie Booking System

Final Project Video Demo Guide

Team: Nguyen Bao Long (523K0014) and Nguyen Ba Hung (523K0006)

Recommended total duration: 8-12 minutes

Overall Recording Plan

- Record each part as a separate short video, then merge them in order.
- Do not show passwords, GitHub Secrets, private keys, .env.production, or SMTP credentials.
- If a login is required, log in before recording or cut out the password entry segment.
- Use browser zoom around 90%-110% and terminal font large enough for the teacher to read.
- Always explain that the selected architecture is Tier 5 k3s Kubernetes, exposed through host Nginx and Kubernetes NodePort because the project runs on a single AWS Learner Lab EC2 host.
- Do not claim Kubernetes Ingress Controller or HPA/autoscaling unless those are implemented.

Video 1 - Introduction And Architecture

Recommended duration: 45-60 seconds

What to show:

Show the report cover page or the architecture/workflow diagram.

Voice-over script:

Hello teacher, this is our final project: CineWorld Movie Booking System.

Our team members are Nguyen Bao Long, student ID 523K0014, and Nguyen Ba Hung, student ID 523K0006.

For this project, we selected Tier 5: Kubernetes-Based Architecture. The application is a microservices movie booking system deployed on an AWS EC2 Ubuntu server using k3s Kubernetes.

The system includes a frontend, multiple FastAPI backend services, MySQL persistent storage, GitHub Actions CI/CD, GHCR container registry, HTTPS domain access, and Prometheus plus Grafana monitoring.

Video 2 - Production Website And HTTPS

Recommended duration: 45-60 seconds

What to show:

Open the public production application and click the HTTPS lock icon.

URL / commands:

`https://tungtungtungtungsahur.site`

Voice-over script:

This is the production website, accessed through our public domain with HTTPS.

The application is not running locally. It is served from the AWS production server. The domain resolves to our Elastic IP, and HTTPS is configured with a valid certificate.

Here we can see the CineWorld frontend loading successfully.

Video 3 - Admin Dashboard And Database

Recommended duration: 60-90 seconds

What to show:

Open the admin dashboard, then open Adminer after login.

URL / commands:

`https://tungtungtungtungsahur.site/admin`

`https://tungtungtungtungsahur.site/adminer/`

Voice-over script:

This is the admin dashboard. It uses the identity service for authentication and the management service for protected admin APIs.

Administrators can manage movies, cinemas, screens, seat types, showtimes, concessions, and revenue reports.

This is Adminer, connected to the production MySQL database. The database contains the application tables such as movies, users, bookings, tickets, transactions, showtimes, seats, and concessions.

Adminer is exposed only for project demonstration and operational evidence. In a long-term production environment, we would restrict or disable this route.

Video 4 - Monitoring With Grafana

Recommended duration: 60-90 seconds

What to show:

Open Grafana and the MovieBooking Final Overview dashboard.

URL / commands:

`https://grafana.tungtungtungtungsahur.site`

Voice-over script:

This is our Grafana monitoring dashboard.

Prometheus collects metrics from the production Kubernetes environment through node-exporter and cAdvisor.

The dashboard shows CPU usage, memory usage, observed containers, and system trends. These metrics reflect the real production server, not a local test environment.

We use this dashboard during deployment verification and failure recovery demonstration.

Video 5 - GitHub Actions CI/CD Overview

Recommended duration: 60-90 seconds

What to show:

Open the GitHub repository, then Actions, then the CI-CD workflow.

Voice-over script:

This is our GitHub Actions CI/CD pipeline.

The pipeline is triggered automatically when code is pushed to the main branch. It contains validation, image build and publish, deployment gate, and production deployment jobs.

The validate job runs static analysis with Ruff, verifies Python syntax, validates Docker Compose, validates Kubernetes deployment scripts, renders k3s manifests, and runs Trivy filesystem security scanning.

For each service, the pipeline builds a Docker image, scans it with Trivy, and pushes the versioned image to GHCR using the Git commit SHA as the image tag.

Video 6 - GHCR Versioned Images

Recommended duration: 30-45 seconds

What to show:

Open GitHub Packages / GHCR packages for the repository owner.

Voice-over script:

Here are the container images published to GHCR.

Each microservice has its own image, for example frontend, catalog, identity, booking, payment, redemption, scheduler, and management.

The images are tagged with the commit SHA. We do not rely only on the latest tag, so the deployment is traceable from source code to production.

Video 7 - Real Code Change

Recommended duration: 60-90 seconds

What to show:

Open VS Code or the local project folder. Edit a small visible frontend text.

URL / commands:

```
Example change in frontend/index.html:

??T V? NGAY

->

??T V? NGAY H?M NAY


git status

git add frontend/index.html

git commit -m "Update homepage call to action for demo"

git push origin main
```

Voice-over script:

Now we demonstrate the mandatory end-to-end CI/CD scenario.

I will make a visible source code change in the frontend. This change will be committed and pushed to the main branch, which triggers the pipeline automatically.

The purpose is to prove that production is updated from source code through CI/CD, not by manual file copying.

The commit has been pushed to the main branch. Now GitHub Actions should start automatically.

Video 8 - CI/CD Run After Push

Recommended duration: 90-150 seconds

What to show:

Open the new GitHub Actions run triggered by the push.

Voice-over script:

This is the new pipeline run triggered by the code push.

First, the validate job checks code quality, Python syntax, Docker Compose configuration, Kubernetes scripts, Kubernetes manifest rendering, and Trivy security scanning.

Then the build-and-publish jobs build service images, scan each image, and push the versioned images to GHCR.

After all images are published, the deploy-production job connects to the EC2 production server, uploads deployment assets, applies the k3s Kubernetes manifests, updates the image tag, waits for rollout completion, and verifies the public HTTPS application and Grafana health endpoint.

After the pipeline completes, all jobs are green, including deploy-production.

Video 9 - Verify Production Update

Recommended duration: 45-60 seconds

What to show:

Refresh the public application after the deployment succeeds.

URL / commands:

```
https://tungtungtungtungsahur.site
```

Voice-over script:

Now we verify the production website after deployment.

The visible text change is now live on the HTTPS production domain. This confirms the full path from source code change, to GitHub Actions CI, to image build and security scan, to automatic Kubernetes deployment, and finally to production verification.

Video 10 - Kubernetes Workloads On EC2

Recommended duration: 60-90 seconds

What to show:

SSH into EC2 and show k3s pods, services, and PVCs.

URL / commands:

```
cd ~/moviebooking-tier5
```

```
sudo k3s kubectl -n moviebooking get pods -o wide
```

```
sudo k3s kubectl -n moviebooking get svc,pvc
```

Voice-over script:

This is the production EC2 server running k3s Kubernetes.

All application pods are running in the moviebooking namespace. We can see the frontend, backend microservices, MySQL, Prometheus, Grafana, node-exporter, and cAdvisor.

The frontend and Grafana are exposed through Kubernetes NodePort services, and persistent volumes are bound for MySQL, Prometheus, and Grafana.

Video 11 - Preflight Check

Recommended duration: 45-60 seconds

What to show:

Run the production preflight script on the EC2 server.

URL / commands:

```
cd ~/moviebooking-tier5
```

```
bash deploy/demo-preflight.sh
```

Voice-over script:

Before failure simulation, we run the production preflight script.

This script validates Kubernetes workloads, local NodePort endpoints, public HTTPS application access, admin dashboard, Adminer, Grafana health, and Prometheus active targets.

The preflight completed successfully, so the production system is healthy before we simulate a failure.

Video 12 - Failure Simulation And Recovery

Recommended duration: 90-120 seconds

What to show:

Run the failure recovery script and optionally open Grafana after recovery.

URL / commands:

```
cd ~/moviebooking-tier5
```

```
bash deploy/demo-simulate-recovery.sh catalog-service https://tungtungtungtungsahur.site
```

Voice-over script:

Now we simulate a production failure by deleting a live pod from the catalog-service deployment.

Because the application runs on Kubernetes, the Deployment controller automatically creates a replacement pod.

The script waits until the new pod is running and ready, then checks the public HTTPS application endpoint again.

This demonstrates Kubernetes self-healing behaviour. The system recovers without manual redeployment.

We can also observe the production system through Grafana after the recovery.

Video 13 - Final Summary

Recommended duration: 30-45 seconds

What to show:

Show the production website, report, or final dashboard.

Voice-over script:

To summarize, our final project implements a production-grade CI/CD system for CineWorld.

The system runs on AWS, uses a public HTTPS domain, deploys microservices to k3s Kubernetes, builds and scans versioned Docker images through GitHub Actions, publishes images to GHCR, deploys automatically to production, and monitors runtime behaviour with Prometheus and Grafana.

We also demonstrated Kubernetes self-healing through pod failure simulation and recovery.

Thank you for watching.

Final Merge Order

1. Introduction And Architecture

2. Production Website And HTTPS

3. Admin Dashboard And Database

4. Monitoring With Grafana

5. GitHub Actions CI/CD Overview

6. GHCR Versioned Images

7. Real Code Change

8. CI/CD Run After Push

9. Verify Production Update

10. Kubernetes Workloads On EC2

11. Preflight Check

12. Failure Simulation And Recovery

13. Final Summary

Final Recording Checklist

- All videos are recorded in the correct order.
- No credentials or private keys are visible.
- The GitHub Actions run shown in the video is real and tied to a commit push.
- The production site is opened through HTTPS domain, not localhost.
- Grafana dashboard shows real production metrics.
- Failure simulation shows pod deletion and Kubernetes recovery.
- The final video and report describe the same deployed system.

End of Video Demo Guide